

Faculty of Science and Engineering

Computer Science

CS-210 Concurrency

May/June 2025

Duration: 2 Hours

Do not turn over your question paper until instructed to do so.

The content of this exam paper has been thoroughly checked, however if you think you have spotted an error, please raise your hand and report it to an invigilator, to log the query. The question will not be corrected in the exam venue. You must also note the query in the answer booklet* and continue to answer the paper to the best of your ability as it is written. Queries will be noted by the module coordinator and the Board of Examiners when it considers results.

*For queries on MCQ papers and Canvas exams, you will be given a separate query slip

Exam Paper Information & Instructions

Answer all questions.

Permitted Materials

Dictionaries – Candidates may only refer to the English and Welsh language dictionaries available

Calculators – Candidates may NOT use a calculator

Open Book – This is NOT an Open Book Exam

Other – None

Additional Materials

None.

1. Please provide the term used to describe the following concepts:

- (a) How do you refer to a statement (or a set of statements) that is executed all at once (physically or logically)? **[2 marks]**
- (b) s is a counting variable that can take only positive integer values. Once s has been given an initial value, it cannot exceed that value. The only allowed operations are $up(s)$ for incrementing s and $down(s)$ for decrementing s . In the context of concurrency, which concept is being captured by s here? **[2 marks]**

2. Using Amdahls law, $L(k, n) \leq \frac{t_s + t_p}{\frac{t_s}{k} + \frac{t_p}{n}}$, answer the following.

- (a) A parallel programme spends 25% of its time in the serial part. Calculate the speedup limit achievable by tripling the serial processing speed after refactoring. **[2 marks]**
- (b) If there were no limits to how many cores we could use to speed up the parallel part, what would be the combined upper limit for the speedup gain? **[2 marks]**
- (c) With a three-time speedup in the serial part, how many cores would you need to achieve a combined speedup of at least 10? **[4 marks]**

3. Two sculptors, tasked with creating independent pieces, were only provided with a chisel and a hammer to share. Each sculptor needs both tools and must wait for availability, as a tool is exclusive to one sculptor at a time.

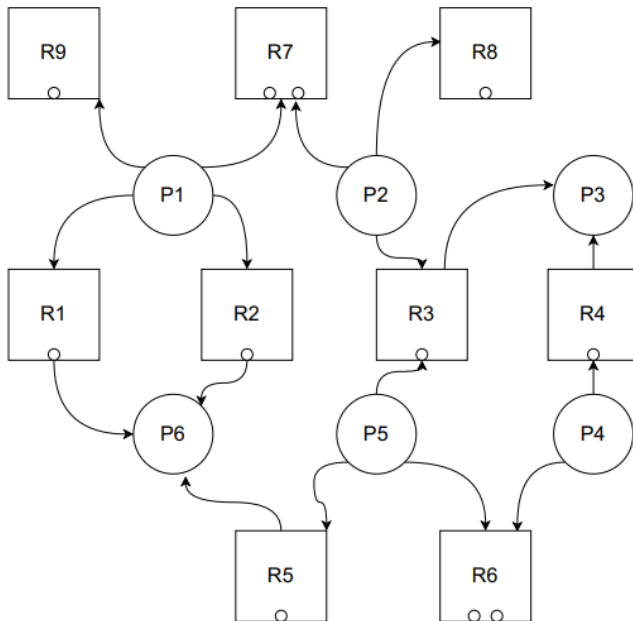
(a) Examine the how this scenario may lead to a deadlock by evaluating each individual Coffman condition. **[4 marks]**

(b) Below is an implementation for the Tools class in the context of this 'Sculptors' scenario. Fix this code within the acquire method to remove deadlocks. **[3 marks]**

```
1 import java.util.concurrent.Semaphore;
2 public class Tools{
3     private Semaphore chisel = new Semaphore(1);
4     private Semaphore hammer = new Semaphore(1);
5     public void acquire(){
6         double r = Math.random();
7         try{
8             if (r>=0.5){
9                 chisel.acquire();
10                hammer.acquire();
11            }else{
12                hammer.acquire();
13                chisel.acquire();
14            }
15        }catch (Exception e){
16            System.out.println("Something went wrong.");
17        }
18    }
19    public void release(){
20        chisel.release();
21        hammer.release();
22    }
23 }
```

(c) Describe which one of the Coffman conditions your solution broke in (c). **[2 marks]**

4. You are given the Resource Allocation Graph (RAG) below.



- For each process, describe whether it will be able to complete its task or not referring to the resources it requires, holds or can hold. **[6 marks]**
- Referring to your analysis in the previous answer, discuss whether this system will lead to deadlock or not. **[2 marks]**
- There are two edges $P1 \rightarrow R2$ and $R2 \rightarrow P6$. How can you change these arrows to cause a potential deadlock in this scenario? **[2 marks]**

5. Below, you are given the code for the monitor class Pot. This was written by an inexperienced programmer for a diner scenario and it is not thread-safe. In this scenario, customers can take a serving, and when the pot is empty, a chef refills the pot.

Your task is to use the multiverse library for software transactional memory (STM) to make the following code thread-safe.

HINTS: you will need to protect the variables, increment them adequately, wrap a relevant method with `atomic(() -> {...})`; and import:

```
import org.multiverse.api.references.*;
import static org.multiverse.api.StmUtils .*;
```

```
1 public class Pot {
2     private int servingCount ;
3     private int capacity ;
4     Pot ( int capacity ){
5         this.capacity = this.servingCount = capacity ;
6     }
7     public void take () throws Exception {
8         if ( servingCount > 0){
9             servingCount -= 1;
10        }
11        else {
12            throw new Exception (" Pot is empty .");
13        }
14    }
15    public void refill () throws Exception {
16        if ( servingCount == 0){
17            servingCount = capacity ;
18        }
19        else {
20            throw new Exception (" Pot is not empty yet.");
21        }
22    }
23 }
```

[5 marks]

6. A *superstore* has an **Entrance** and an **Exit**. The policy is that **at most 3 shoppers will be allowed in a store at any point in time**. Therefore, the supermarket has deployed an assistant to count how many customers come in, and only allow someone to enter if the store is open and there are fewer than 3 customers in the store. On the other hand, if the store has been closed, then the assistant only allows people to leave. The manager usually instructs the assistant to open or close the store at any time. Naturally, no one can leave if the store is empty.

(a) Write the FSP of a simple process *Entrance* that models the enter action of a customer entering the superstore. **[1 mark]**

(b) Write the FSP of simple process *Exit* that models the leave action of a customer leaving the superstore. **[1 mark]**

(c) Write the FSP of simple process *Manager* whose task is to open and close the shop. **[2 marks]**

(d) Write the FSP of an *Assistant* process counting customers to allow them to enter and leave and open and close the superstore accordingly, as described. A skeleton is provided below to help you with this task.

```
Assistant(N=3) = Closed[0],  
Closed[count:0..N] = (when count>0 ... | ...),  
Opened[count:0..N] = (when count>0 ... | when count<N ...  
                      | ...).
```

[5 marks]

(e) Write the FSP of the resulting *SuperStore* process. **[3 marks]**

(f) Sketch the resulting LTS diagram. **[2 marks]**